Vietnamese Traffic Sign Detection and Warning System

Hoang Gia Bao

Phan Boi Chau High School for the Gifted, Nghe An, Vietnam

Email: hgbao0907@gmail.com

Abstract

Accurate, real-time traffic sign recognition is a cornerstone of Advanced Driver Assistance Systems (ADAS) and autonomous vehicles, especially on resource-constrained edge devices – yet Vietnam's traffic environment has lacked datasets and pipelines robust enough to reflect it realistically. This work presents a leakage-free training and evaluation pipeline for the VR-TSD dataset, built around three technical contributions. First, near-duplicate frames are detected by combining filename provenance with 256-bit perceptual hashing, merged into clusters by union-find, and the 8,078 images are split 70/15/15 at the cluster level, so that no image has a counterpart in another subset within the hash threshold. Second, an audit of the 58-category taxonomy shows that three categories have no test instance and fourteen have fewer than ten, so test-set class-averaged metrics are computed over the 55 categories present in the test split; a bounding-box-aware offline augmentation of 21 rare or hard classes is evaluated against a control run without it, which attains a lower clean-test mAP@0.5 (95.34% versus 96.17%) and a markedly lower mAP@0.5 at the heaviest fog (74.45% versus 80.16%) and rain levels (78.53% versus 84.57%). Third, robustness is measured on two synthetic domains – procedural haze and windshield-raindrop rendering – each at four fixed severity levels. Trained at 896×896 resolution, the YOLO11n detector (2.59 M parameters, 5.6 MB) reaches, over three training seeds, a clean-test mAP@0.5 of 96.17% ± 0.32 (mAP@0.5:0.95 = 79.67% ± 0.70) and a Macro F1 of 93.11% ± 0.48 at a fixed confidence threshold of 0.25; mAP@0.5 remains above 90% up to the third fog level (93.11%) and the second rain level (91.39%). Paired with an audio-based warning module, a Streamlit web application, and on-device inference at 78.74 FPS on a Samsung Galaxy S23, the system is a feasible candidate for ADAS deployment tailored to Vietnam's traffic conditions, pending in-vehicle validation.

Introduction

Traffic sign detection and recognition is a crucial component of Advanced Driver Assistance Systems (ADAS), supporting traffic safety and enabling various levels of autonomous driving. These systems require both high accuracy and low latency to operate reliably in diverse real-world environments. While vehicle and pedestrian detection often demand high frame rates (≥30 FPS), traffic sign recognition – being relatively static – typically requires only about 10 FPS (<100 ms latency) to ensure timely driver assistance.

Recent research has applied and optimized object detection models such as YOLO, Faster R-CNN, and MobileNet-SSD for traffic sign detection, especially for small and distant signs under challenging conditions. However, most existing studies rely on foreign datasets or limited Vietnamese datasets that lack diversity in sign types, environmental conditions, and traffic scenarios. As a result, current models may not generalize well to the complex characteristics of Vietnamese highways, national roads, and urban traffic.

This study addresses these limitations by proposing a lightweight and efficient traffic sign detection system tailored for Vietnam, with a particular focus on the reliability of the data itself rather than only the model. The main contributions include:

1. **Leakage-free, cluster-level data split.** I design a duplicate screening pipeline – filename-provenance matching combined with 256-bit perceptual-hash near-duplicate detection – and merge matching images into clusters by union-find over all 8,078 images. The images are then split 70/15/15 by multi-label stratified sampling applied to whole clusters, so that every cluster lies in exactly one subset; no image has a counterpart in another subset within the hash threshold or shares a source identifier with it.

2. **Class-coverage audit of the full 58-class taxonomy.** All 58 categories are retained. Per-class instance counts over the three subsets identify the categories that cannot be evaluated reliably (three have no test instance, fourteen have fewer than ten), and test-set class-averaged metrics are computed over the 55 categories present in the test split. Five low-frequency categories are learned by the full model, and a 53-class variant without them does not improve the remaining classes.

3. **Offline rare/hard-class augmentation with a measured effect.** A bounding-box-aware Albumentations pipeline multiplies 21 under-represented or difficult classes toward a target instance count in the training split

only. A controlled comparison shows that it raises clean-test mAP@0.5 by 0.83 points and mAP@0.5 at the heavier fog and rain levels by 3.0 to 6.0 points.

4. **Fixed-severity stress-test domains.** Two offline evaluation domains – procedural haze and windshield-raindrop rendering – are generated from the held-out test split at four severity levels each. The levels are fixed before evaluation and are not tuned to the model's response, so the degradation curves characterize the detector rather than a selected operating point.

5. **Edge-optimized, single-model pipeline.** A lightweight YOLO11n model is trained end-to-end on the cleaned and augmented data with three training seeds, evaluated with a single matching procedure for all domains (no test-time augmentation, 95% cluster-bootstrap intervals), and validated on real hardware – reaching 78.74 FPS on a Samsung Galaxy S23 at a 5.6 MB footprint.

6. **End-to-end system.** The trained model is packaged into a Streamlit-based web application (image and video input, no installation required) with a concise audio warning module that conveys the essential meaning of each detected sign to the driver.

Related work

Traffic sign detection is a fundamental problem in computer vision and plays an important role in Advanced Driver Assistance Systems (ADAS) and autonomous vehicles. With the rapid development of deep learning, Convolutional Neural Network (CNN)-based methods have achieved significant improvements in object detection performance. Early two-stage detectors, such as R-CNN and its successors, provided high detection accuracy but suffered from relatively high computational complexity and slow inference speed. To address these limitations, one-stage detectors, particularly the YOLO (You Only Look Once) family, were introduced to achieve a better balance between detection accuracy and inference efficiency.

The YOLO family has undergone continuous architectural improvements over successive generations. Early versions, from YOLOv1 to YOLOv3, established the foundation for real-time object detection and introduced multi-scale feature representations to improve the detection of small objects. Later generations, including YOLOv5, further improved computational efficiency and provided lightweight variants suitable for deployment on resource-constrained devices. YOLOv8 introduced a modern anchor-free detection architecture and further improvements in both accuracy and inference efficiency. More recently, YOLOv10 incorporated architectural optimizations aimed at reducing inference latency and computational overhead, while YOLO11 further improved the trade-off between accuracy, model complexity, and deployment efficiency. Building on this progress, Ultralytics introduced YOLO26, which removes the Non-Maximum Suppression (NMS) post-processing step and the Distribution Focal Loss (DFL) module in favor of a native end-to-end prediction pipeline, together with training refinements aimed at improving small-object detection and inference speed on edge and low-power hardware [15].

Among lightweight YOLO variants, YOLO11n offers a favorable accuracy/size/speed trade-off for edge deployment; at a compact 5.6 MB with 2,593,462 parameters, it is adopted as the target architecture for all of the data-pipeline and robustness work described in the remainder of this report.

Another important limitation of existing traffic sign datasets is their inability to fully represent the diverse and challenging environmental conditions encountered in real-world Vietnamese traffic. Publicly available datasets, such as GTSRB and TT100K, primarily focus on traffic environments from Germany and China, respectively, and may not adequately capture local characteristics or adverse visual conditions in Vietnam. Beyond environmental diversity, a second and less-discussed limitation of Roboflow-style exported datasets is that train/validation/test splits are frequently generated from video-extracted frames without explicit duplicate control, which can silently leak near-identical frames across splits and inflate reported generalization performance. This study treats both problems – environmental diversity and cross-split duplication – as first-class concerns rather than only tuning model architecture.

Dataset and Proposed Model

Base dataset

The original dataset was obtained from the Vietnamese Real-World Traffic Sign Dataset (VR-TSD), which contains 58 traffic sign categories commonly encountered in Vietnam. [12] The dataset consists of 8,078 images with 13,016 manually annotated bounding boxes, released with an original split of 70% training (5,668 images), 16% validation (1,259 images), and 14% test (1,151 images). On average, each image contains one to three traffic signs, while some images contain up to five signs. The released split is not used in this study: all 8,078 images are pooled and re-partitioned at the cluster level, as described next.

Leakage screening and cross-split consolidation

During development on the released split, a discrepancy of 9.38 percentage points was observed between validation and test mAP@0.5, which was substantially larger than would be expected from independent held-out subsets and raised concerns about duplicate or near-duplicate frames crossing the validation/test boundary – a plausible risk given that the source images were extracted from dashcam video. A dedicated screening and clustering pipeline was therefore built, rather than merely re-shuffling the data.

Duplicate detection. Two complementary detectors are applied to the pooled images: (i) a filename-provenance check that recovers the shared Roboflow source identifier embedded in each filename and links images that trace back to the same source frame; and (ii) a perceptual-hash check, using a 256-bit DCT-based hash (16×16 low-frequency coefficients thresholded at their median) with a Hamming-distance threshold of 16, which additionally links near-duplicate frames that do not share a filename (e.g., adjacent frames from the same video clip). A coarser 8×8 average hash with a threshold of 4 proved non-discriminative: it matches 3,101 of the 5,668 released training images (54.7%) to a validation or test image, and still reports 2,575 train–validation matches after the cluster-level split described below. The 256-bit hash at threshold 16 matches 584 of them (10.3%; 666, or 11.8%, together with the filename check). The number of matches increases gradually with the threshold, from 303 at distance 0 to 772 at distance 32, without a distinct plateau.

Cluster-level stratified split. Matching images are merged into clusters by a union-find procedure over all 8,078 images, yielding 6,813 clusters, of which 387 contain more than one image (the largest holds 148 images, 1.8% of the data). Of these, 191 clusters span more than one subset of the released split, so the released partition does not keep near-duplicate frames apart. Iterative multi-label stratified sampling is then applied to clusters, each described by the set of classes it contains and its image count, with target proportions of 70/15/15, so that every cluster is assigned entirely to one subset. The resulting subsets contain 5,471 training (67.7%), 1,341 validation (16.6%), and 1,266 test images (15.7%), with 2,114 and 1,971 annotated instances in validation and test; the deviation from the target proportions arises because clusters are indivisible. Verification on the final split finds no image with a counterpart in another subset within the hash threshold and no shared source identifier (0 matches for each of the three subset pairs). Unless otherwise noted, all results reported in this study (Tables 2–6 and Figures 1–4) are computed on this split.

Class-taxonomy audit

All 58 categories of the source taxonomy are retained. Per-class instance counts over the three subsets (Figure 1) show that three categories (131b-cam-do-xe-ngay-le, 301e-cac-xe-chi-duoc-re-trai-2, and 301b-cac-xe-chi-duoc-re-phai, with two training instances each) have no test instance and therefore cannot be evaluated; they remain in the 58-class label space during training, while test-set class-averaged metrics are computed over the 55 categories with at least one test instance. Thus, 58 denotes the number of classes in the main model, and 55 denotes the number of classes represented in the test split; no class is removed from the main model. Fourteen of these 55 categories have fewer than ten test instances (72 of 1,971 instances, 3.7%), so their per-class scores carry little statistical weight.

Five low-frequency categories (133-het-cam-vuot, 421-ket-thuc-khu-dong-dan-cu, 423a-duong-nguoi-di-bo-sang-ngang-1, 124b-cam-oto-quay-dau-xe, and 420-bat-dau-khu-dong-dan-cu), which had the lowest recall in preliminary runs on the released split, were examined separately. In the 58-class model they reach a mean test F1 of 0.951 and a mean AP@0.5 of 0.987 over 98 test instances. A separate 53-class ablation variant without them, trained under the same protocol, gives a mAP@0.5 of 96.24% on the test split, against 95.94% for the 58-class model on the 50 test categories common to both. The five categories are therefore learnable on the cluster-level split, and removing them does not improve the remaining ones.

Offline rare- and hard-class augmentation

The taxonomy still exhibits strong class imbalance. I flag any class with fewer than 50 training instances as rare (19 classes), and separately designate three classes that had proven historically difficult in prior training runs (134-het-han-che-toc-do-toi-da, 103b-cam-oto-re-phai, 301a-cac-xe-chi-duoc-di-thang) as hard, regardless of their instance count; the union comprises 21 classes. For every training image containing at least one rare or hard class, I generate additional bounding-box-aware augmented copies using a targeted Albumentations pipeline (shift/scale/rotate, brightness/contrast, hue-saturation-value, and Gaussian blur, with a minimum bounding-box visibility of 0.3), with the number of copies per image scaled inversely to that image's rarest class count (bounded between 3 and 20 copies) so as to approach a target of 100 instances per class without over-multiplying already-adequate classes. This procedure generated 1,970 synthetic training images covering the 21 rare/hard classes, applied to the training split only.

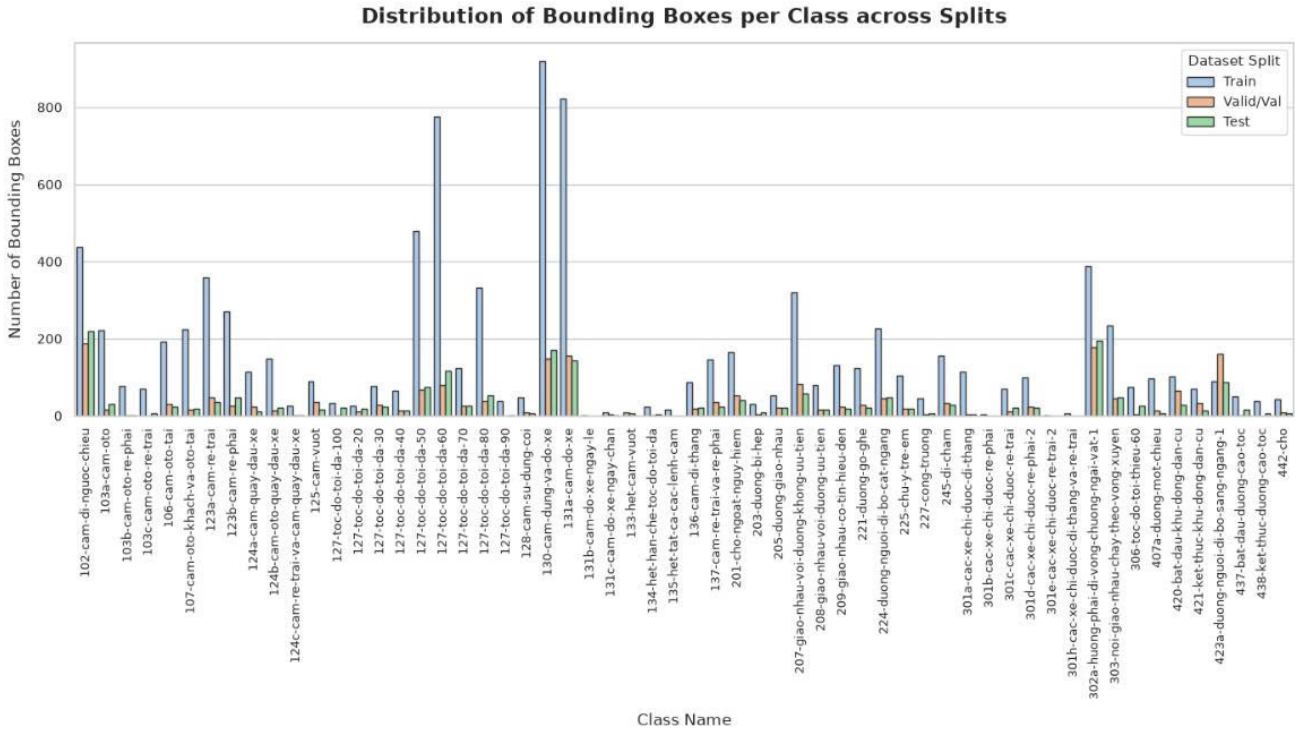


Figure 1: Number of Bounding Boxes of each label in VR-TSD (58 classes, cluster-level split)

Methodology

The experimental pipeline was organized into ten stages: dataset loading, duplicate screening and clustering, cluster-level stratified split, class-taxonomy audit, offline rare/hard-class augmentation, stress-test domain generation, model training, confidence-threshold selection, multi-domain evaluation, and export/deployment. Experiments were conducted in a Kaggle environment with dual NVIDIA T4 GPU acceleration.

Dataset Loading: The environment was initialized with a fixed random seed (2102) across Python, NumPy, and PyTorch for the data split, the augmentation, and the stress-test generation. The three released folders (training, validation, test) were pooled for re-partitioning, as described in the Dataset section above.

Leakage Screening and Split: Section "Leakage screening and cross-split consolidation" above details the filename- and hash-based duplicate detection, the union-find clustering of all images, and the cluster-level multi-label stratified split.

Class Audit and Rare-Class Augmentation: The full 58-class label space was retained after the audit described above; bounding-box-aware Albumentations augmentation was then applied to rare and historically hard classes in the training split only.

Stress-Test Domain Generation (Evaluation Only): Two offline stress-test domains – Fog and Rain – were generated once from the held-out test split (1,266 images), each at four severity levels (F1–F4 and R1–R4, Table 1), and were never seen during training. The levels were fixed before the reported evaluation and were not adjusted using its results. Fog uses a procedural haze model: a low-resolution random noise field is Gaussian-blurred and alpha-blended with a uniform haze color, with intensity randomly drawn per image from the range [0.25, 0.50] on top of a base haze thickness that increases over the four levels (0.10, 0.25, 0.40, 0.55) and a haze color sampled in [205, 228] (grayscale). It resembles the persistent urban haze and dust commonly observed in major Vietnamese cities rather than dense, opaque fog, and it does not model depth-dependent attenuation. Rain uses a windshield-raindrop renderer (external raindrops.py module) that simulates Snell refraction, Fresnel edge highlights, and gravity-fed rivulets rather than simple overlaid streaks. Severity increases with the number of drops, their maximum radius, the probabilities of large drops and streaks, the number of rivulets, and the fraction of the frame height they extend over; the fourth level uses the default parameters of the renderer.

Model Training: A single YOLO11n model was trained from the COCO-pretrained checkpoint on the cleaned, augmented 58-class training set, using only standard Ultralytics online augmentation (mosaic, HSV jitter, translate/scale) – no synthetic weather effect is injected during training, so that the Fog and Rain stress-test domains remain a genuinely held-out robustness check rather than a distribution the model was exposed to. Training was repeated with three seeds (2102, 42, 2201) on the same split, and a control model without the offline augmentation was trained once (seed 2102) for the ablation.

Confidence Threshold Selection: Three policies were compared: (i) a fixed threshold of 0.25; (ii) a single global threshold that maximizes Macro F1 on the validation subset; and (iii) per-class thresholds that

Table 1: Severity levels of the stress-test domains

Level	Parameters
Fog F1–F4	base haze thickness 0.10 / 0.25 / 0.40 / 0.55
Rain R1	60 drops, radius 8–16 px, large drops 3% (22–32 px), streaks 15%, 8 rivulets, height 80%
Rain R2	150 drops, radius 8–20 px, large drops 7% (26–40 px), streaks 30%, 20 rivulets, height 85%
Rain R3	175 drops, radius 8–22 px, large drops 10% (27–43 px), streaks 35%, 23 rivulets, height 87%
Rain R4	200 drops, radius 8–24 px, large drops 12% (28–46 px), streaks 40%, 26 rivulets, height 89%

maximize each class's F1 on the validation subset, with classes having fewer than 15 validation instances retaining 0.25 since their F1 curves are too noisy to calibrate reliably. Thresholds were searched over [0.05, 0.95] in steps of 0.01 and were never fitted on the test subset or on the stress-test domains; the global threshold is applied to the stress-test domains.

Multi-Domain Evaluation: Each model was evaluated on the validation subset, the clean test subset, and the eight stress-test domains, with an IoU threshold of 0.5 for matching. mAP@0.5 and mAP@0.5:0.95 were computed with the Ultralytics validator at a confidence of 0.001; Precision, Recall, and F1 were computed with a single matching routine (greedy assignment by descending confidence) applied identically to all domains, without test-time augmentation, and averaged over the classes that have at least one ground-truth instance in the evaluated subset. Results are reported as mean ± standard deviation over the three training seeds, and 95% confidence intervals were obtained by bootstrap resampling of image clusters (1,000 resamples).

Export and Deployment: The trained model was exported to ONNX for portability, and separately to TorchScript/NCNN for on-device benchmarking; it was integrated into a Streamlit-based web application supporting image and video input with visual detection overlays and concise audio warnings.

Training and Deployment Environment

All training experiments were conducted on Kaggle using two NVIDIA T4 GPUs. The final model is based on YOLO11n, trained at 896×896 resolution to better preserve features of small, distant traffic signs. Table 2 summarizes the training configuration; the learning rate is scaled with total batch size (capped at 0.004) and cosine-annealed to a final fraction of 0.01. Deterministic random seeds were used throughout to support reproducibility.

Table 2: Hyperparameters for model training (YOLO11n, this study)

Category	Specification	Value
Training environment	GPU	2 × NVIDIA T4
	Image size (imgsz)	896
Training parameters	Maximum epochs	200
	Batch size (total)	64
	Optimizer	AdamW
	Patience (early stopping)	35
	Initial learning rate (lr0)	0.0020
	Final LR fraction (lrf)	0.01
	LR schedule	Cosine
	Warm-up epochs	8
	Box / Cls / DFL loss gain	7.5 / 0.6 / 1.5
	Mosaic (close at epoch)	1.0 (15)
	Translate / Scale	0.05 / 0.5
	HSV (h/s/v)	0.01 / 0.5 / 0.3
	Horizontal / Vertical flip	0.0 / 0.0

For real-world evaluation, the model was deployed via the Ultralytics app on two representative smartphones: the Samsung Galaxy S23 (upper high-end) and the Google Pixel 5 (upper mid-range). The model was exported to TorchScript and subsequently converted into the NCNN framework for mobile deployment; input videos were processed at 1080p resolution. GPU throughput was measured on the same NVIDIA T4 ×2 (Kaggle) environment used for training, reported here alongside the on-device figures for transparency and reproducibility.

Experimental Results

Overall Performance

Table 3 reports the performance of the YOLO11n model across the evaluation domains as mean ± standard deviation over three training seeds. On the clean test split the model reaches mAP@0.5 = 96.17% ± 0.32 and mAP@0.5:0.95 = 79.67% ± 0.70; at the global confidence threshold selected on validation (0.19, 0.12, and 0.32 for the three seeds), Precision, Recall, and Macro F1 are 90.54%, 95.07%, and 92.34%. All three runs exceed the design targets (mAP@0.5 ≥93%, Macro F1 ≥90%); the 95% cluster-bootstrap interval of the clean-test Macro F1 is [90.1, 93.6], [88.8, 92.6], and [91.1, 94.6] for the three seeds, so its lower bound falls below 90% in one of them. The validation subset yields lower scores (mAP@0.5 = 94.20% ± 0.61; Macro F1 = 87.24% ± 1.61, at a threshold fitted on that subset) than the test subset. This is consistent with its composition: it contains a large concentration of sign 420 (144 of its 194 instances) and several categories with only one or two instances, whose AP is unstable.

Table 3: YOLO11n performance across evaluation domains (this study, 58-class model; mean ± s.d. over three seeds; Precision, Recall, and Macro F1 at the validation-selected global threshold)

Domain	mAP@50	mAP@50-95	Precision	Recall	Macro F1
Validation	94.20 ± 0.61	77.20 ± 0.36	87.24 ± 0.13	91.56 ± 1.56	87.24 ± 1.61
Test (Clean)	96.17 ± 0.32	79.67 ± 0.70	90.54 ± 2.39	95.07 ± 0.65	92.34 ± 1.13
Fog F1	95.95 ± 0.30	79.46 ± 0.32	89.57 ± 1.91	94.49 ± 0.14	91.49 ± 0.98
Fog F2	95.28 ± 0.33	78.56 ± 0.40	88.92 ± 2.50	93.72 ± 0.36	90.79 ± 1.22
Fog F3	93.11 ± 0.15	75.64 ± 0.27	87.06 ± 2.23	89.22 ± 2.04	87.40 ± 0.23
Fog F4	80.16 ± 1.97	63.80 ± 1.52	80.66 ± 2.97	74.76 ± 4.26	75.84 ± 1.48
Rain R1	94.89 ± 0.55	77.48 ± 0.80	90.04 ± 2.15	92.66 ± 0.90	90.92 ± 0.87
Rain R2	91.39 ± 0.73	71.10 ± 0.50	86.98 ± 2.83	88.06 ± 1.74	86.94 ± 0.99
Rain R3	87.10 ± 0.16	67.14 ± 0.48	84.18 ± 3.76	82.66 ± 1.43	82.27 ± 1.43
Rain R4	84.57 ± 0.24	63.20 ± 0.34	83.22 ± 3.88	79.25 ± 2.30	80.35 ± 0.83

The main model is trained with all 58 classes. The clean test split and its Fog/Rain derivatives contain ground-truth instances for 55 classes; their class-averaged metrics therefore cover these 55 classes. The three classes absent from the test split are retained in the model. The separate 53-class variant is an ablation, not the main model.

With a fused footprint of 2,593,462 parameters, 101 layers, and 5.6 MB on disk (PyTorch weights; 10.3 MB after ONNX export), the model remains well suited to memory- and resource-constrained edge deployment while retaining strong clean-domain accuracy.

Table 4: On-device inference throughput of the deployed YOLO11n model. GPU throughput measured on the NVIDIA T4 ×2 (Kaggle) training environment; Google Pixel 5 and Samsung Galaxy S23 measured on-device via the Ultralytics app (NCNN backend).

Platform	ms / image	FPS
NVIDIA T4 ×2 (Kaggle)	5.78	173.01
Google Pixel 5	58.0	17.24
Samsung Galaxy S23	12.7	78.74

Confidence-threshold policy. Table 5 compares the three threshold policies on the clean test split. Their Macro F1 values differ by at most 1.2 points (93.11% ± 0.48 for the fixed threshold, 92.34% ± 1.13 for the global threshold, 93.54% ± 0.12 for per-class thresholds), which is of the order of the seed-to-seed spread. The global threshold is the least stable: its validation optimum varies between 0.12 and 0.32 across seeds, indicating a shallow F1 optimum. Per-class thresholds yield the smallest variance and favor Precision (94.28%) over Recall (93.54%), but they are fitted on few validation instances for many classes and do not outperform the fixed threshold by a margin exceeding the seed spread. mAP@0.5, which does not depend on the threshold, is therefore used as the primary accuracy measure.

Contribution of offline augmentation. Table 6 compares the full pipeline with a control model trained without the offline augmentation. Removing it lowers clean-test mAP@0.5 by 0.83 points (2.6 standard deviations of the three-seed spread) and validation mAP@0.5 by 2.2 points, and lowers mAP@0.5 at the heavier stress levels (F3, F4, R3, R4) by 3.0 to 6.0 points, well beyond the seed spread of 0.15 to 1.97 points at these levels; at the fixed threshold, Macro F1 differs by 0.5 points, within the seed spread. The loss is not confined to the targeted classes: on the validation subset, mAP@0.5 of the 18 rare or hard classes present in it falls from 93.3% to 91.4%, and that of the remaining classes from 94.6% to 92.3%. As the augmentation pipeline also contains photometric

Table 5: Threshold policies on the clean test split (mean ± s.d. over three seeds)

Policy	Precision	Recall	Macro F1	Micro F1
Fixed (0.25)	91.89 ± 0.66	95.12 ± 0.05	93.11 ± 0.48	91.91 ± 0.28
Global (from validation)	90.54 ± 2.39	95.07 ± 0.65	92.34 ± 1.13	91.27 ± 1.28
Per-class (from validation)	94.28 ± 0.07	93.54 ± 0.28	93.54 ± 0.12	92.85 ± 0.09

and blur transformations, the gain plausibly reflects greater image diversity as well as class balancing; these components were not separated, and the control was run once.

Table 6: Effect of offline augmentation (with: mean ± s.d. over three seeds; without: single run, seed 2102)

Metric	With aug.	Without aug.	Difference
Test mAP@50	96.17 ± 0.32	95.34	−0.83
Test mAP@50-95	79.67 ± 0.70	78.10	−1.57
Validation mAP@50	94.20 ± 0.61	92.00	−2.20
Test Macro F1 (fixed 0.25)	93.11 ± 0.48	92.58	−0.53
Fog F3 mAP@50	93.11 ± 0.15	89.51	−3.60
Fog F4 mAP@50	80.16 ± 1.97	74.45	−5.71
Rain R3 mAP@50	87.10 ± 0.16	84.08	−3.02
Rain R4 mAP@50	84.57 ± 0.24	78.53	−6.04

Training Behavior

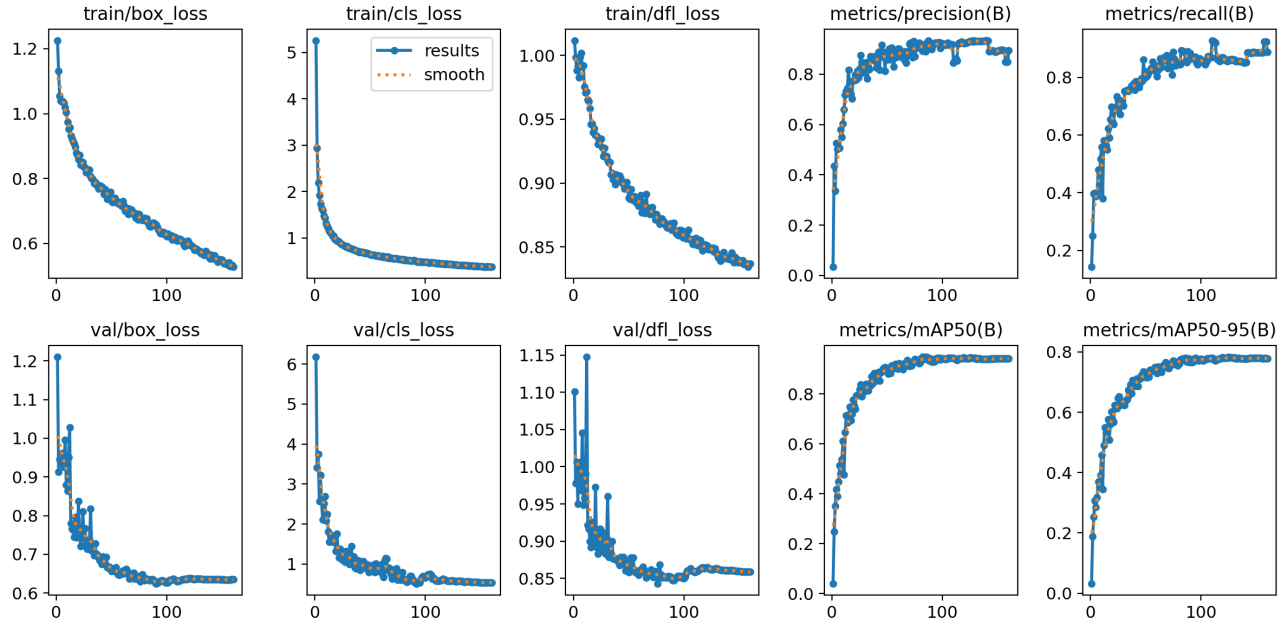


Figure 2: YOLO11n training curves (this study, seed 2102) – box/cls/DFL loss and Precision/Recall/mAP per epoch.

Training and validation losses (box, classification, and DFL) decrease steadily across epochs with no sign of divergence, while Precision, Recall, mAP@0.5, and mAP@0.5:0.95 rise quickly in the early epochs and converge toward the domain-level figures reported in Table 3, indicating a stable optimization process without evidence of severe overfitting. Early stopping (patience 35) ended training after 144 to 184 epochs in the three runs (best epochs 109 to 149; 3.6 to 4.8 hours on two T4 GPUs), before the 200-epoch limit. The precision-recall curve on the validation split (Figure 3) closes at an overall mAP@0.5 of 0.935 for the seed-2102 model, consistent with the validation figures in Table 3.

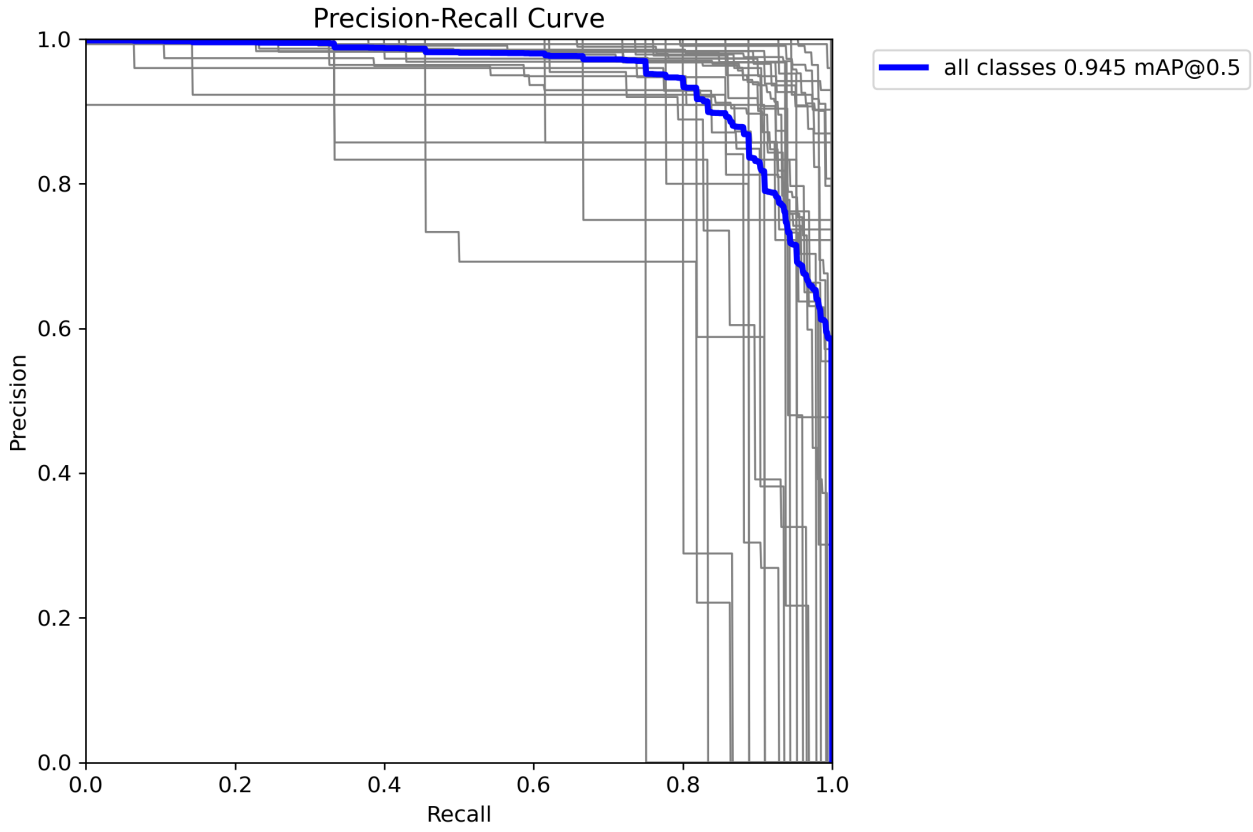


Figure 3: Precision-Recall curve on the validation split (seed 2102, all classes, mAP@0.5 = 0.935).

Robustness Under Fog and Rain

Table 3 and Figure 4 report performance at the four severity levels of each domain. mAP@0.5 decreases monotonically with severity: for fog from 95.95% ± 0.30 (F1) and 95.28% ± 0.33 (F2) to 93.11% ± 0.15 (F3) and 80.16% ± 1.97 (F4); for rain from 94.89% ± 0.55 (R1) and 91.39% ± 0.73 (R2) to 87.10% ± 0.16 (R3) and 84.57% ± 0.24 (R4), against 96.17% on clean images. Under light haze (F1, F2) the loss relative to clean images is below 1 point (0.22 and 0.89); it reaches 3.06 points at F3 and 16.01 points at F4. Under rain the corresponding losses are 1.28, 4.78, 9.07, and 11.60 points. Macro F1 follows the same ordering (92.34% clean; 87.40% at F3 and 75.84% at F4; 86.94% at R2 and 80.35% at R4). The same ordering holds for the control model without offline augmentation, with larger losses (Figure 4, Table 6).

The degradation is dominated by Recall rather than Precision. From clean images to F4, Recall decreases from 95.07% to 74.76% (20.3 points) while Precision decreases from 90.54% to 80.66% (9.9 points); from clean images to R4 the decreases are 15.8 points (to 79.25%) and 7.3 points (to 83.22%). Because the confidence threshold is selected on clean validation images, part of the recall loss reflects lower confidence of correct detections; the threshold-independent mAP@0.5 decreases in the same direction, so the degradation is not an artefact of the threshold. Under rain, mAP@0.5:0.95 declines faster than mAP@0.5 at every level (R2: 8.6 versus 4.8 points; R4: 16.5 versus 11.6 points), whereas under fog the two decline at the same rate at F4 (15.9 versus 16.0 points); this is consistent with raindrop refraction and rivulets distorting object boundaries more than a global photometric haze does.

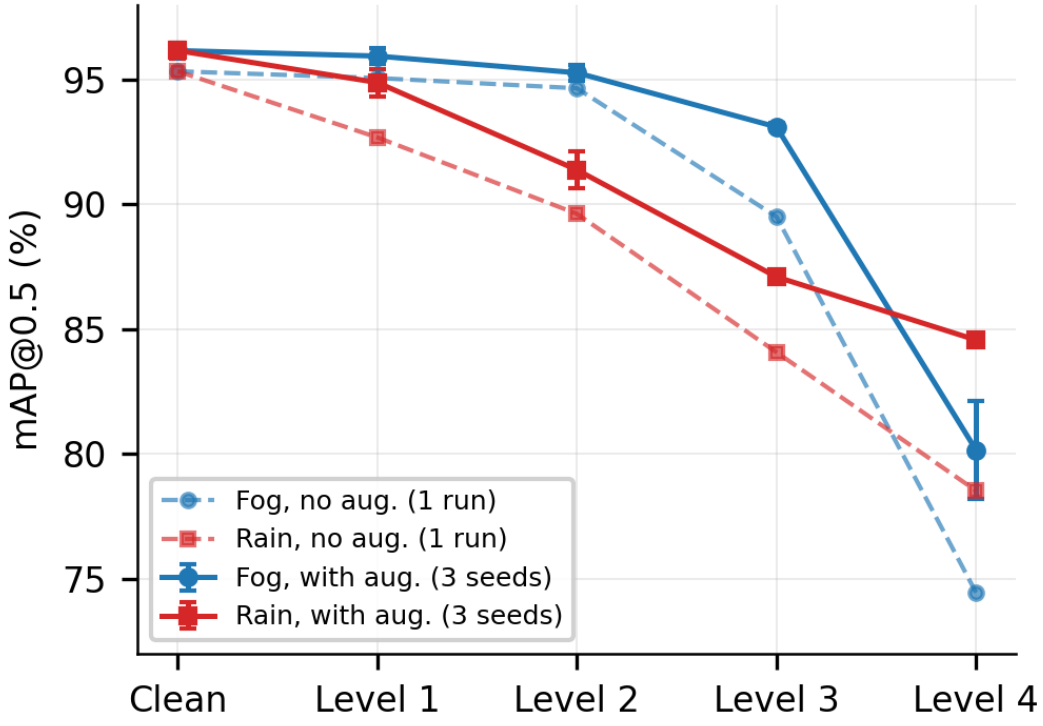


Figure 4: mAP@0.5 versus severity level for fog and rain (with augmentation: mean ± s.d. over three seeds; without augmentation: single run).

Conclusion

This study delivers a leakage-controlled and more reproducible traffic sign recognition pipeline for Vietnam's road conditions, whose contribution lies in the data protocol and its evaluation rather than in a new architecture. A single YOLO11n detector – trained on a cluster-split, offline-augmented version of VR-TSD with all 58 categories – reaches, over three training seeds, a clean-test mAP@0.5 of 96.17% ± 0.32 (mAP@0.5:0.95 = 79.67% ± 0.70) and a Macro F1 of 93.11% ± 0.48 at a fixed threshold of 0.25, at a 5.6 MB footprint. Under four fixed severity levels per domain, mAP@0.5 remains above 90% up to F3 (93.11%) and R2 (91.39%) and decreases to 80.16% at F4 and 84.57% at R4. With 17.24 and 78.74 FPS on the Google Pixel 5 and the Samsung Galaxy S23, both above the roughly 10 FPS required for traffic sign recognition, and a Streamlit-based web application with audio warnings, these results support the feasibility of the system for ADAS use on edge devices.

These results rest on three measured interventions. Duplicate control: clustering the 8,078 images produced 6,813 clusters and showed that 191 of them spanned more than one subset of the released split; the cluster-level split leaves no cross-subset match within the hash threshold, whereas the coarser 8×8 average hash flagged 54.7% of the released training images and was not discriminative. Class coverage: retaining all 58 categories costs nothing in accuracy, since the five low-frequency categories reach a mean test F1 of 0.951 and a 53-class variant does not improve the remaining classes (96.24% versus 95.94% mAP@0.5 on the common categories); the audit also identifies the categories that cannot be evaluated. Augmentation: the control without offline augmentation loses 0.83 points of clean-test mAP@0.5 and 3.0 to 6.0 points at the heavier stress levels. Because the stress levels were fixed in advance, the degradation curves describe the detector itself, and the threshold comparison shows that F1 depends on the threshold policy by up to 1.2 points, so mAP@0.5 is the more reliable summary.

The following limitations bound these conclusions and define the next iteration. First, the evaluation covers 55 of 58 categories, and 14 of them have fewer than ten test instances; the 95% cluster-bootstrap interval of the clean-test Macro F1 spans up to 3.8 points (for example [88.8, 92.6] for seed 2201), and residual errors concentrate in visually similar speed-limit signs (for the seed-2102 model, 5 of 27 "70 km/h" instances were predicted as "60 km/h" and 4 of 60 "80 km/h" instances likewise). Second, the cluster-level split leaves some classes unevenly distributed across subsets (sign 420: 41, 144, and 9 instances in training, validation, and test), which makes validation harder than test and the validation-selected thresholds sensitive to single scenes; a larger source pool for the rare categories is needed rather than another re-shuffle of the existing one. Third, the hash threshold of 16 has no distinct plateau in the match counts, so residual dependence between frames of the same video separated by more than this distance cannot be excluded, and all results refer to the VR-TSD distribution. Fourth, only three training seeds were used, and the augmentation control was run once; its components (class balancing,

photometric and blur transformations) were not separated. Fifth, the Fog and Rain domains are synthetic – the haze model has no depth dependence and the rain renderer is an external script – and real low-light, foggy, and rainy driving footage is not yet covered. Sixth, the on-device throughput was measured on an earlier export of the same YOLO11n architecture with a 53-class head; the 58-class model differs only in the width of the classification head, and the audio warning module has not been evaluated for latency or false alarms. The next iteration will therefore add real adverse-condition footage, more data for the rare categories, re-benchmarking of the 58-class export, and on-device testing on Raspberry Pi and Jetson Nano.

References

[1] Yang, Y., Luo, H., Xu, H., & Wu, F. (2015). Towards real-time traffic sign detection and classification. IEEE Transactions on Intelligent transportation systems, 17(7), 2022-2031.

[2] Sarvajcz, K., Ari, L., & Menyhart, J. (2024). Ai on the road: nvidia jetson nano-powered computer vision-based system for real-time pedestrian and priority sign detection. Applied Sciences, 14(4), 1440.

[3] Yang, M., & Han, S. (2025). ETS-YOLO: An Efficient YOLO-based Model for Real-Time Traffic Sign Recognition. Signal, Image and Video Processing, 19(8), 650.

[4] Chaudhuri, A. (2024). Smart traffic management of vehicles using faster R-CNN based deep learning method. Scientific Reports, 14(1), 10357.

[5] Asif, S. M., Anil, T., Tangudu, S., & Keertan, G. S. (2023, May). Traffic Sign Detection Using SSD Mobilenet & Faster RCNN. In 2023 2nd International Conference on Vision Towards Emerging Trends in Communication and Networking Technologies (ViTECoN) (pp. 1-6).

[6] Huan, T. P., & Lenh, P. P. C. (2024, November). Vietnamese Traffic Sign Detection for Advanced Driving Assistant: Comparison Between YOLOv8 and Faster RCNN. In International Conference on Advances in Information and Communication Technology (pp. 617-628). Cham: Springer Nature Switzerland.

[7] Vennelakanti, A., Shreya, S., Rajendran, R., Sarkar, D., Muddegowda, D., & Hanagal, P. (2019, January). Traffic sign detection and recognition using a CNN ensemble. In 2019 IEEE international conference on consumer electronics (ICCE) (pp. 1-4). IEEE.

[8] Wang, Z., Qu, M., Wang, N., Liu, L., & Su, H. (2025). Lightweight traffic sign detection algorithm based on improved YOLOv5 in snowy environments. Signal, Image and Video Processing, 19(5), 1-11.

[9] Jiang, P., Ergu, D., Liu, F., Cai, Y., & Ma, B. (2022). A Review of Yolo algorithm developments. Procedia computer science, 199, 1066-1073.

[10] Xuan, Y., Zhang, X. Y., Li, C., Wang, H., & Mu, C. (2025). LAM-YOLOv11 for UAV transmission line inspection: Overcoming environmental challenges with enhanced detection efficiency. Multimedia Systems, 31(2), 1-13.

[11] Abdi, N., Parvaresh, F., & Sabahi, M. F. (2024). SeqNet: Sequential networks for one-shot traffic sign recognition with transfer learning. IEEE Transactions on Intelligent Transportation Systems.

[12] Ngoc, D. N., Nhat, L. N. H., Van, Q. H., Nguyen, H., Bui, H., & Nguyen, P. A. (2026). VR-TSD: A Real-World Dataset and a Lightweight YOLOv8n Model for Traffic Sign Recognition in Vietnam. In J. S. Park, D. Camacho, S. Gritzalis, & J. J. H. Park (Eds.), *Proceedings of CSA 2025*. Lecture Notes in Electrical Engineering, vol. 1583, pp. 105–115. Springer, Singapore. `https://link.springer.com/chapter/10.1007/978-981-95-7896-2_10`. Publisher page.

[13] Source code. `https://github.com/HoangGiaBao107/Vietnamese-Traffic-Sign-Detection-and-Warning-System`.

[14] Link website. `https://vietnamese-traffic-sign-detection-and-warning-system-hgb.streamlit.app/`.

[15] Ultralytics. YOLO26 – Model documentation. `https://docs.ultralytics.com/models/yolo26/`.